

Maximum Bipartite Matching

Samarth Singla

September 15, 2026

What is a Matching?

Matching (M): A set of edges in a graph where no two edges share the same vertex. Everyone gets at most one partner.

Maximal vs. Maximum (The Classic Trap)

- **Maximal Matching:** You cannot add any more edges without breaking the rules. You are "stuck" locally.
- **Maximum Matching:** The absolute largest possible matching in the entire graph. The global optimum.

Note: Every Maximum matching is Maximal, but a Maximal matching is not necessarily Maximum!

Key Definitions: Vertices and Paths

To find the Maximum matching, we need a few definitions relative to our current matching M :

- **Matched Vertex:** A vertex connected to an edge in M .
- **Free Vertex:** A vertex not connected to any edge in M .
- **Alternating Path:** A path whose edges strictly alternate between being in M and outside of M .
- **Augmenting Path:** A specific type of alternating path that **starts and ends with a free vertex**.

Lemma 1: The "Flip" ($M \oplus P$)

Lemma 1: If P is an M -augmenting path, we can create a strictly larger matching by taking the symmetric difference: $M \oplus P$.

Why does this work? (The Proof Intuition)

- An augmenting path starts and ends with free vertices, meaning its first and last edges are **outside** M .
- Therefore, it has odd length: $k + 1$ edges outside M , and k edges inside M .
- If we "flip" the states of all edges in P (edges in M are removed, edges outside M are added), the new matching is completely valid.
- Since we added $k + 1$ edges and removed k edges, the new matching size is exactly $|M| + 1$.

Comparing Two Matchings

Question: How do we prove we can *always* reach the maximum?

Suppose our matching M is not maximum, and a larger optimal matching M^* exists ($|M^*| > |M|$). Look at their symmetric difference: $M \oplus M^*$.

- Every vertex has at most one edge from M and one from M^* .
- Max degree is 2, meaning the graph breaks down purely into isolated **cycles** and **paths**.
- Edges in these components strictly alternate between M and M^* .
- **The Deduction:** Cycles and even-length paths have an equal number of M and M^* edges. For M^* to be strictly larger, there **must** be an odd-length path starting and ending in M^* .
- *Conclusion:* This path is exactly an M -augmenting path!

Berge's Lemma

The previous logic directly gives us the most important theorem for matching.

Berge's Lemma

A matching M is a maximum cardinality matching **if and only if** it has no augmenting path.

Algorithm Strategy (Naïve Iterative Scheme):

- 1 Start with an empty matching $M = \emptyset$.
- 2 Find an augmenting path P .
- 3 Update matching: $M \leftarrow M \oplus P$ (Flip the path).
- 4 Repeat until no augmenting paths exist. By Berge's Lemma, we stop at the Maximum Matching!

Finding Augmenting Paths

How do we code a search for an augmenting path? We turn it into a directed graph problem (G_M).

The Rules of Graph G_M :

- Orient every **unmatched** edge from Left to Right ($L \rightarrow R$).
- Orient every **matched** edge from Right to Left ($R \rightarrow L$).

The Result: An augmenting path is simply a directed path from a **free Left vertex** to a **free Right vertex**. We can easily find this using standard DFS!

C++ Implementation: Kuhn's Algorithm DFS

```
1 // u: current left vertex
2 // adj: adjacency list (only left to right edges)
3 // mr: match of right vertex (-1 if free)
4 // used: visited array to prevent infinite loops
5
6 bool try_kuhn(int u, vector<vector<int>>& adj,
7               vector<int>& mr, vector<bool>& used) {
8     for (int v : adj[u]) {
9         if (used[v]) continue;
10        used[v] = true;
11
12        // If right vertex 'v' is free, OR
13        // its current match can find a different partner
14        if (mr[v] == -1 || try_kuhn(mr[v], adj, mr, used)) {
15            mr[v] = u; // Flip the edge!
16            return true;
17        }
18    }
19    return false;
20 }
```

C++ Implementation: Main Loop & Complexity

```
1 int max_matching(int L, int R, vector<vector<int>>& adj) {  
2     vector<int> mr(R, -1);  
3     int matches = 0;  
4  
5     for (int u = 0; u < L; u++) {  
6         vector<bool> used(R, false);  
7         if (try_kuhn(u, adj, mr, used)) {  
8             matches++;  
9         }  
10    }  
11    return matches;  
12 }
```

Time Complexity:

- The main loop runs exactly once per vertex in the Left set (V).
- Each DFS search scans at most $O(E)$ edges.
- Overall Time Complexity: $O(V \cdot E)$

Concept Check: Perfect Matchings

Problem 1: k -regular Bipartite Graphs

Let $G = (L, R, E)$ be a bipartite graph where every vertex has exactly degree k ($k \geq 1$).

Prove that G always contains a perfect matching.

Hint: You may use Hall's Marriage Theorem, which states that a bipartite graph has a matching completely covering L if and only if for every subset $S \subseteq L$, the size of its neighborhood $|N(S)| \geq |S|$.

Solution 1: Double Counting

Proof using Hall's Condition:

Let $S \subseteq L$ be any arbitrary subset of vertices in L . We want to show $|N(S)| \geq |S|$.

- 1 Count the edges leaving S . Since every vertex has degree k , there are exactly $k \cdot |S|$ edges originating from S .
- 2 All these edges must connect to vertices in $N(S)$ (by definition of a neighborhood).
- 3 The maximum number of edges the set $N(S)$ can "receive" is $k \cdot |N(S)|$, because each vertex in R also has a strict degree of k .
- 4 Therefore, the total number of edges from S cannot exceed the total capacity of $N(S)$:

$$k \cdot |S| \leq k \cdot |N(S)|$$

- 5 Dividing both sides by k , we get $|S| \leq |N(S)|$.

Since Hall's condition holds for all possible subsets S , a perfect matching is guaranteed to exist. ■

Concept Check: Edge Partitioning

Problem 2: Disjoint Matchings

Using the result established in Problem 1, prove the following statement:

If $G = (L, R, E)$ is a k -regular bipartite graph, its edges can be partitioned into exactly k mutually disjoint perfect matchings.

Discussion Point: Think about how you would prove this step-by-step. How does finding just one perfect matching change the state of the graph?

Solution 2: Proof by Induction

Proof via Induction on k :

Base Case ($k = 1$): The graph is 1-regular. Every vertex connects to exactly one other vertex. The edges themselves already form exactly 1 perfect matching.

Inductive Step:

- 1 Assume the statement holds for some degree $k - 1$.
- 2 Consider a k -regular bipartite graph G . By Problem 1, we know G has at least one perfect matching. Let's extract it and call it M_1 .
- 3 Remove all edges belonging to M_1 from G .
- 4 Since M_1 is a perfect matching, we just removed exactly one edge from *every single vertex* in the graph.
- 5 The remaining graph G' is now exactly $(k - 1)$ -regular.
- 6 By our inductive hypothesis, G' can be cleanly partitioned into $k - 1$ disjoint perfect matchings.

Together with M_1 , we have partitioned E into exactly k perfect matchings.

